CS152 Assignment 2: The 8-puzzle

Before you turn in this assignment, make sure everything runs as expected. First, **restart the kernel** (in the menubar, select Kernel→ Restart) and then run the test cells for each of the questions you have answered. Note that a grade of 3 for the A* implementation requires all tests in the "Basic Functionality" section to be passed. The test cells pass if they execute with no errors (i.e. all the assertions are passed).

Make sure you fill in any place that says `YOUR CODE HERE` . Be sure to remove the `raise NotImplementedError()` statements as you implement your code - these are simply there as a reminder if you forget to add code where it's needed.

Question 1

Define your `PuzzleNode` class below. Ensure that you include all attributes that you need to implement an A* search. If you wish, you can even include member functions, such as a function to generate successor states. Alternatively, you can code up this functionality later in the `solvePuzzle` function.

In [1]:

```
# Import any packages you need here
# Also define any variables as needed

# YOUR CODE HERE (OPTIONAL)
import numpy as np
import copy

#Now, define the class PuzzleNode:
class PuzzleNode:
    """
    Class PuzzleNode: Provides a structure for performing A* search for the n^2-1 puzzle

    Attributes:
        puzzle (numpy array): Current state of a puzzle board as a numpy array
        state (list of list): A List of List that identifies the current state of a puzzle board
        parent (PuzzleNode): A PuzzleNode that has aleardy been expanded into successor nodes
        depth (int): The depth of a PuzzleNode from the root node
        f_value (int): The evaluation function of a specific node
        pruned (bool): Boolean to identify whether a node has been pruned or not
        length (int): The size/rows of the Puzzle board which is mostly identical to the column
```

```
    """
    def __init__(self, state, depth, parent):
        """
        Constructor of the PuzzleNode Class

        Params:
            state (list of list): A List of List that identifies the current state of a puzzle board
            depth (int): The depth of a PuzzleNode from the root node
            parent (PuzzleNode): A PuzzleNode that has aleardy been expanded into successor nodes
        """


        # used to easily change it to string
        self.puzzle = np.array(state)
        self.state = self.puzzle.tolist()
        self.parent = parent
        self.depth = depth
        self.f_value = 0
        self.length = len(self.state)
        self.pruned = False

    def __lt__(self, other):
        """
        A method that helps the PriorityQueue compare the evaluation function between PuzzleNodes
        """
        return self.f_value < other.f_value

    def __str__(self):
        """
        A method that helps turn puzzle board lists to strings
        """
        return self.puzzle.__str__()

    def empty_tile(self):
        """
        A method to find the position of an empty tile
        """
        for x in range(len(self.state)):
            for y in range(len(self.state)):
                if self.state[x][y] == 0:
                    return x,y

    def copy(self):
        """
        A method to copy a PuzzleNode and connect self as a parent node
```

```
        """
        #use deepcopy to get an exact replica
        temp_state = copy.deepcopy(self.state)
        #retruns Puzzle node with new replica,
        #set parent as self, and increase depth by 1
        return PuzzleNode(temp_state, self.depth+1, self)

    def successor_states(self):
        """
        A method to generate successor states by swapping the empty tile with its adjacent tiles.

        Returns:
            list: A list of PuzzleNode objects that represent the successor states.
        """
        # Find the position of the empty tile.
        empty_x, empty_y = self.empty_tile()

        # List the four possible directions to swap the empty tile.
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

        # Generate the successor states by swapping the empty tile with its adjacent tiles.
        successor_states = []
        for dx, dy in directions:
            x, y = empty_x + dx, empty_y + dy
            if 0 <= x < self.length and 0 <= y < self.length:
                # Create a copy of the current state and swap the tiles.
                new_state = copy.deepcopy(self.state)
                new_state[empty_x][empty_y], new_state[x][y] = new_state[x][y], new_state[empty_x][empty_y]
                successor_node = PuzzleNode(new_state, self.depth + 1, self)
                successor_states.append(successor_node)

        return successor_states
```

Question 2

Define your heuristic functions using the templates below. Ensure that you extend the `heuristics` list to include all the heuristic functions you implement. Note that state will be given as a list of lists, so ensure your function accepts this format. You may use packages like numpy if you wish within the functions themselves.

In [38]:

```
# Add any additional code here (e.g. for the memoization extension)
#extension for memoiazation
```

```python
from itertools import permutations

def memoize(h):
    memo = {}
    def memoized_func(state):
        if state in memo:
            return memo[state]
        memo[state] = h(state)
        return memo[state]
    return memoized_func

def goal_state(state):
    """
    A function that determines the desired goal state for a puzzle board in a given state
    """
    length = len(state)
    goal_board = [[j + length * i for j in range(length)] for i in range(length)]
    return goal_board

# Misplaced tiles heuristic
def h1(state):
    """
    This function returns the number of misplaced tiles, given the board state
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the number of misplaced tiles
    """
    length = len(state)
    goal_board = goal_state(state)
    num_misplaced = 0

    for i in range(length):
        for j in range(length):
            if state[i][j] != goal_board[i][j] and state[i][j] != 0:
                num_misplaced += 1

    return num_misplaced

# Manhattan distance heuristic
def h2(state):
    """
    This function returns the Manhattan distance from the solved state, given the board state
    Input:
        -state: the board state as a list of lists
```

```
    Output:
        -h: the Manhattan distance from the solved configuration
    """
    n = len(state)
    manhattan_distance = 0
    for i in range(n):
        for j in range(n):
            if state[i][j] != 0:
                row, col = divmod(state[i][j], n)
                manhattan_distance += abs(row - i) + abs(col - j)
    return manhattan_distance


# Memoized Misplaced tiles heuristic
@memoize
def h1_memoized(state):
    """
    This function returns the number of misplaced tiles, given the board state
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the number of misplaced tiles
    """
    length = len(state)
    goal = goal_state(state)
    misplaced = 0

    for i in range(length):
        for j in range(length):
            if state[i][j] != goal[i][j] and state[i][j] != 0:
                misplaced += 1
    return misplaced

# Memoized Manhattan distance heuristic
@memoize
def h2_memoized(state):
    """
    This function returns the Manhattan distance from the solved state, given the board state
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the Manhattan distance from the solved configuration
    """
    length = len(state)
    goal = goal_state(state)
```

```
    manhattan_distance = 0

    for i in range(length):
        for j in range(length):
            if state[i][j] !=  0:
                x = state[i][j]%length
                y = state[i][j]//length
                manhattan_distance += abs(x-j) + abs(y-i)

    return manhattan_distance

# Extra heuristic for the extension.  If implemented, modify the function below

def h3(state):
    """
    This function returns a heuristic value for the given board state that outperforms the Manhattan distance
    heuristic in terms of the number of nodes expanded while maintaining the same optimal number of steps.
    Input:
        -state: the board state as a list of lists
    Output:
        -h: the heuristic value
    """
    n = len(state)
    misplaced_tiles = 0
    row_conflicts = 0
    col_conflicts = 0

    # Calculate the number of misplaced tiles and the number of row and column conflicts
    for i in range(n):
        for j in range(n):
            if state[i][j] == 0:
                continue
            row_goal, col_goal = divmod(state[i][j] - 1, n)
            if i != row_goal or j != col_goal:
                misplaced_tiles += 1
                if i == row_goal:
                    col_conflicts += 1
                if j == col_goal:
                    row_conflicts += 1

    # Calculate the Manhattan distance for each tile
    manhattan_distance = 0
    for i in range(n):
        for j in range(n):
            if state[i][j] == 0:
```

```
                continue
            row_goal, col_goal = divmod(state[i][j] - 1, n)
            manhattan_distance += abs(row_goal - i) + abs(col_goal - j)

    # Add the number of misplaced tiles and conflicts to the Manhattan distance to get the final heuristic value
    h = manhattan_distance + 2 * (row_conflicts + col_conflicts) + misplaced_tiles
    return h
```

Question 3

Code up your A* search using the SolvePuzzle function within the template below. Please do not modify the function header, otherwise the automated testing will fail. You may define other functions or import packages as needed in this cell or by adding additional cells.

Extension Questions

The extensions can be implemented by modifying the code from Q2-3 above appropriately.

1. **Initial state solvability:** Modify your SolvePuzzle function code in Q3 to return -2 if an initial state is not solvable to the goal state.
2. **Extra heuristic function:** Add another heuristic function (e.g. pattern database) that dominates the misplaced tiles and Manhattan distance heuristics to your Q2 code.
3. **Memoization:** Modify your heuristic function definitions in Q2 by using a Python decorator to speed up heuristic function evaluation

There are test cells provided for extension questions 1 and 2.

In [13]:

```
# Import any packages or define any helper functions you need here
from queue import PriorityQueue

#function to check the validity of input puzzle
def is_valid(state):
    """
    Function to check the validity of input puzzle
    """
    row_length = len(state)
    column_length = len(state[0])
    flattened_state = []

    #provides a flattened array
    for lst in state:
        flattened_state += lst
```

```
    goal_state = [i for i in range(1, row_length**2)] + [0]

    #check if the input is empty
    if row_length == 0:
        return False
    #check if each row is the same length
    for lst in state:
        if len(lst) != row_length:
            return False

    #check for duplicate values in the puzzle
    for tile in goal_state[::-1]:
        if tile in flattened_state:
            goal_state.pop()

    if goal_state != []:
        return False

    return True


# This A* search algorithm is mainly dependent on the class implementation from session
# Main solvePuzzle function.
def solvePuzzle(state, heuristic):
    """This function should solve the n**2-1 puzzle for any n > 2 (although it may take too long for n > 4)).
    Inputs:
        -state: The initial state of the puzzle as a list of lists
        -heuristic: a handle to a heuristic function.  Will be one of those defined in Question 2.
    Outputs:
        -steps: The number of steps to optimally solve the puzzle (excluding the initial state)
        -exp: The number of nodes expanded to reach the solution
        -max_frontier: The maximum size of the frontier over the whole search
        -opt_path: The optimal path as a list of list of lists.  That is, opt_path[:,:,i] should give a list of lists
                    that represents the state of the board at the ith step of the solution.
        -err: An error code.  If state is not of the appropriate size and dimension, return -1.  For the extention task,
          if the state is not solvable, then return -2
    """

    goal = np.array(goal_state(state)).reshape(len(state), len(state)).tolist()

    #initilizaing output variables
    steps = 0
    exp = 0
    max_frontier = 0
```

```
    opt_path = 0
    err = 0
    #count the number of elements in the frontier
    cur_frontier = 0

    #check the validity of the given puzzle
    if not is_valid(state):
        err = -1
        return steps,exp,max_frontier,opt_path, err

    #use PriorityQueue data structure for our frontier
    frontier = PriorityQueue()
    #initilaze our puzzleNode
    start = PuzzleNode(state, 0, None)

    #initializing the evaluation to the heuristic value
    start.f_value = start.depth + heuristic(start.state)

    #Use dictionary data structure to keep track of visited states
    #avoids looping and running forever
    visited = dict()
    visited[str(start.state)] = start

    frontier.put(start)
    cur_frontier += 1


    while not frontier.empty():
        #keeps track of maximum frontier
        if cur_frontier > max_frontier:
            max_frontier = cur_frontier

        #pops the node with the least f_value
        cur_node = frontier.get()

        cur_frontier -= 1

        #skip pruned nodes
        if cur_node.pruned:
            continue
        #reached goal state, terminiate loop
        if cur_node.state == goal:
            break

        else:
```

```
            children = cur_node.successor_states()

            exp += 1

            for child in children:
                if str(child.state) in visited.keys():
                    if visited[str(child.state)].depth > child.depth:
                        visited[str(child.state)].pruned = True
                    else:
                        continue

                #update the evaluation function
                child.f_value = child.depth + heuristic(child.state)


                frontier.put(child)
                cur_frontier += 1
                visited[str(child.state)] = child

    opt_path = [cur_node.state]

    #trace back the goal state to the start
    while cur_node.parent:
        opt_path.append((cur_node.parent).state)
        cur_node = cur_node.parent

    #reverses the order of the path so the initial comes first
    opt_path = opt_path[::-1]
    steps = len(opt_path) - 1
```

Basic Functionality Tests

The cells below contain tests to verify that your code is working properly to be classified as basically functional. Please note that a grade of **3** on #aicoding and #search as applicable for each test requires the test to be successfully passed. **If you want to demonstrate some other aspect of your code, then feel free to add additional cells with test code and document what they do.**

In [4]:

```
## Test for state not correctly defined

incorrect_state = [[0,1,2],[2,3,4],[5,6,7]]
_,_,_,_,err = solvePuzzle(incorrect_state, lambda state: 0)
assert(err == -1)
```

In [5]:

```
## Heuristic function tests for misplaced tiles and manhattan distance

# Define the working initial states
working_initial_states_8_puzzle = ([[2,3,7],[1,8,0],[6,5,4]], [[7,0,8],[4,6,1],[5,3,2]], [[5,7,6],[2,4,3],[8,1,0]])

# Test the values returned by the heuristic functions
h_mt_vals = [7,8,7]
h_man_vals = [15,17,18]

for i in range(0,3):
    h_mt = heuristics[0](working_initial_states_8_puzzle[i])
    h_man = heuristics[1](working_initial_states_8_puzzle[i])
    assert(h_mt == h_mt_vals[i])
    assert(h_man == h_man_vals[i])
```

In [6]:

```
## A* Tests for 3 x 3 boards
## This test runs A* with both heuristics and ensures that the same optimal number of steps are found
## with each heuristic.

# Optimal path to the solution for the first 3 x 3 state
opt_path_soln = [[[2, 3, 7], [1, 8, 0], [6, 5, 4]], [[2, 3, 7], [1, 8, 4], [6, 5, 0]],
                 [[2, 3, 7], [1, 8, 4], [6, 0, 5]], [[2, 3, 7], [1, 0, 4], [6, 8, 5]],
                 [[2, 0, 7], [1, 3, 4], [6, 8, 5]], [[0, 2, 7], [1, 3, 4], [6, 8, 5]],
                 [[1, 2, 7], [0, 3, 4], [6, 8, 5]], [[1, 2, 7], [3, 0, 4], [6, 8, 5]],
                 [[1, 2, 7], [3, 4, 0], [6, 8, 5]], [[1, 2, 0], [3, 4, 7], [6, 8, 5]],
                 [[1, 0, 2], [3, 4, 7], [6, 8, 5]], [[1, 4, 2], [3, 0, 7], [6, 8, 5]],
                 [[1, 4, 2], [3, 7, 0], [6, 8, 5]], [[1, 4, 2], [3, 7, 5], [6, 8, 0]],
                 [[1, 4, 2], [3, 7, 5], [6, 0, 8]], [[1, 4, 2], [3, 0, 5], [6, 7, 8]],
                 [[1, 0, 2], [3, 4, 5], [6, 7, 8]], [[0, 1, 2], [3, 4, 5], [6, 7, 8]]]

astar_steps = [17, 25, 28]
for i in range(0,3):
    steps_mt, expansions_mt, _, opt_path_mt, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[0])
    steps_man, expansions_man, _, opt_path_man, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[1])
    # Test whether the number of optimal steps is correct and the same
    assert(steps_mt == steps_man == astar_steps[i])
    # Test whether or not the manhattan distance dominates the misplaced tiles heuristic in every case
    assert(expansions_man < expansions_mt)
    # For the first state, test that the optimal path is the same
    if i == 0:
        assert(opt_path_mt == opt_path_soln)
```

In [9]:
```
## A* Test for 4 x 4 board
## This test runs A* with both heuristics and ensures that the same optimal number of steps are found
## with each heuristic.

working_initial_state_15_puzzle = [[1,2,6,3],[0,9,5,7],[4,13,10,11],[8,12,14,15]]
steps_mt, expansions_mt, _, _, _ = solvePuzzle(working_initial_state_15_puzzle, heuristics[0])
steps_man, expansions_man, _, _, _ = solvePuzzle(working_initial_state_15_puzzle, heuristics[1])
# Test whether the number of optimal steps is correct and the same
assert(steps_mt == steps_man == 9)
# Test whether or not the manhattan distance dominates the misplaced tiles heuristic in every case
assert(expansions_mt >= expansions_man)
```

Extension Tests

The cells below can be used to test the extension questions. Memoization if implemented will be tested on the final submission - you can test it yourself by testing the execution time of the heuristic functions with and without it.

In [15]:
```
## Puzzle solvability test

unsolvable_initial_state = [[7,5,6],[2,4,3],[8,1,0]]
_,_,_,_,err = solvePuzzle(unsolvable_initial_state, lambda state: 0)
assert(err == -2)
```

In [39]:
```
## Extra heuristic function test.
## This tests that for all initial conditions, the new heuristic dominates over the manhattan distance.

dom = 0
for i in range(0,3):
    steps_new, expansions_new, _, _, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[2])
    steps_man, expansions_man, _, _, _ = solvePuzzle(working_initial_states_8_puzzle[i], heuristics[1])
    # Test whether the number of optimal steps is correct and the same
    assert(steps_new == steps_man == astar_steps[i])
    # Test whether or not the manhattan distance is dominated by the new heuristic in every case, by checking
    # the number of nodes expanded
    dom = expansions_man - expansions_new
    assert(dom > 0)
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
<ipython-input-39-5b5de863ad21> in <module>
```

```
     11     # the number of nodes expanded
     12     dom = expansions_man - expansions_new
---> 13     assert(dom > 0)

AssertionError:
```

In [ ]:

```
## Memoization test - will be carried out after submission
```